DOCKER CONTAINERS AND CONTAINER ISOLATION

A Report on Containerization and Its Role in Cloud Computing

Submitted by: **AKASH RAJEEV SHRIVASTAVA**

Register Number: **24BCE7650**

Course / Subject: **OPERATING SYSTEMS**

Faculty Name: **MONALISA SAHU**

Institution: **VIT AP UNIVERSITY**

Topic: Docker Containers & Cloud Computing — Container Isolation

Abstract

An application that runs perfectly on one developer's laptop can fail on another machine because of differences in runtime versions, installed libraries and system configuration. Docker addresses this problem by packaging an application together with its runtime and dependencies so that it can run in a consistent environment on other compatible machines.

This report explains what Docker is, how a Dockerfile becomes an image and an image becomes a running container, and how Docker differs from GitHub. It then focuses on container isolation — how containers share one host while keeping separate views of processes, files and networks using Linux namespaces and control groups — and compares containers with virtual machines. A simple Node.js project is used as a running example.

Objectives

- Explain the problem Docker solves using a simple Node.js example.
- Define Docker, Dockerfiles, images and containers.
- Differentiate Docker from GitHub.
- Describe container isolation and the technologies behind it.
- Compare containers with virtual machines.
- Identify practical applications of Docker in modern software development.

1. Introduction — The Problem Docker Solves

Let us begin with a simple example. Suppose a developer builds a Node.js project on a laptop and it works perfectly. The project is then shared with a friend, but when the friend tries to run it on his own laptop, it produces errors.

Why does this happen? There are several possible reasons:

- **Different runtime versions:** the developer may have Node.js version 20 while the friend has version 18.
- **Missing libraries:** certain packages installed on one machine may not exist on the other.
- **Different configuration:** operating-system settings and environment variables may not match.

GitHub can be used to share the source code, but GitHub alone does not guarantee that the project will run correctly on another machine. Code is only one part of what an application needs. This is where Docker comes in.

2. What Is Docker?

Docker is a platform that helps developers build, package, distribute and run applications inside containers. It provides tools to create images, start and manage containers, connect containers through networks, and store persistent data in volumes.

2.1 The lunchbox analogy

Think of Docker like a lunchbox. Instead of carrying only the food, we prepare everything needed and pack it together so that it is ready to use. Similarly, Docker packages an application along with its runtime and dependencies so that it can run in a consistent environment on another compatible machine.

IN SHORT

Docker does not just share the code — it shares the **environment** the code needs in order to run.

3. Docker Images and Containers

To understand Docker, three terms are essential: Dockerfile, image and container.

- **Dockerfile:** a text file containing the instructions for preparing the application environment.
- **Docker image:** the packaged template created from those instructions. It contains the application files and the required dependencies.
- **Container:** a running instance of an image.

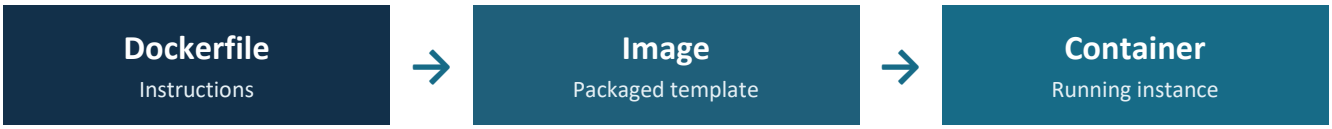


Figure 1. The basic Docker workflow

3.1 The blueprint analogy

A Docker image is like the blueprint of a house, while a container is an actual house built from that blueprint. Just as many houses can be built from one blueprint, many containers can be created from the same image.

3.2 Core Docker terms

Term	Meaning	Analogy
Dockerfile	Text file containing instructions to build an image.	Recipe or assembly instructions.
Image	Read-only template containing application files and required dependencies.	Blueprint of a house.
Container	A running instance of an image, with its own runtime configuration.	A house built from the blueprint.
Registry	A service for storing and distributing images, such as Docker Hub.	A warehouse for packaged kits.
Docker Engine	The software that builds images and manages containers.	The machinery that builds and runs them.
Volume	Storage that persists data beyond a container's writable layer.	A separate storage cabinet.

3.3 Example: packaging the Node.js project

In the Node.js example, the developer creates a Dockerfile, builds an image from it, and runs a container from that image. The image can then be published to a container registry so that the friend can download and run it.

```
FROM node:20
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

FROM selects the Node.js base image, WORKDIR sets the working directory, COPY transfers project files, RUN installs the dependencies while the image is built, EXPOSE documents the application port, and CMD defines the startup command.

```
docker build -t my-project .
docker run --name my-app -p 3000:3000 my-project
docker ps
docker stop my-app
docker images
```

These commands build an image, start a container with host port 3000 mapped to container port 3000, list running containers, stop the container and list local images. Images should be versioned, and passwords or secret keys should never be embedded in them.

4. Docker Versus GitHub

It is natural to ask whether GitHub already does something similar. It does not. GitHub is mainly used to store source code, track changes and collaborate with other developers. Docker is used to package the application environment and run it inside containers.

Aspect	GitHub	Docker
Main purpose	Hosting source code and tracking changes with Git.	Packaging the application environment and running it in containers.
What it stores	Source code, Dockerfiles, documentation.	Images (through a registry) and running containers.
Helps the friend to…	Download the code.	Run the code in a consistent environment.
Collaboration	Reviews, branches, pull requests.	Shared, versioned images.

> **KEY POINT**
> GitHub and Docker **complement each other rather than replace each other**. GitHub lets the friend download the code; Docker helps him run it.

5. Container Isolation

Container isolation means that applications running in different containers have separate views of selected system resources, reducing interference between them. Containers can share the same host computer while maintaining separate process, filesystem and network environments.

5.1 The hostel analogy

Imagine several students living in the same hostel. They share the same building, but each student has a separate room and personal belongings. Similarly, containers share the same host while each has its own environment. The analogy is helpful, but real isolation is enforced by operating-system mechanisms rather than walls.

5.2 Main areas of isolation

Area	What it means	Example
Process isolation	A container sees its own processes rather than every process on the host.	One web service cannot automatically inspect another container's processes.
Filesystem isolation	Each container has its own root filesystem view and writable layer.	Packages installed in one container do not appear in another.
Network isolation	Network interfaces, addresses and rules can be kept separate.	A service is reachable only on an intended network or published port.
Resource control	CPU and memory can be limited and monitored.	A memory limit stops one application from consuming all host memory.

5.3 Technologies behind isolation

On Linux, Docker relies on two key technologies:

- **Namespaces** separate the views of resources such as processes, networks, mount points and host names.
- **Control groups (cgroups)** monitor and limit resources such as CPU and memory.

For example, if two web applications run in separate containers, their environments can be configured independently and each can be given its own memory limit.

5.4 Isolation is not perfect security

Container isolation is a useful layer of separation, but it is not a guarantee of security. In the usual Linux container model, containers share the host kernel, so a kernel vulnerability or an unsafe configuration can affect the host. Proper configuration and good security practices therefore remain important — for instance, using trusted images, avoiding unnecessary privileges and keeping secrets out of images.

6. Containers Versus Virtual Machines

Another important comparison is between containers and virtual machines. A virtual machine generally runs its own guest operating system on virtualized hardware. A container, in the usual Linux model, shares the host kernel and isolates the application using operating-system features.

Feature	Containers	Virtual machines
Operating-system kernel	Usually share the host kernel.	Each VM runs its own guest OS.
Startup time	Generally fast, since a container is an isolated process.	Generally slower, since a guest OS must boot.
Resource usage	Generally lower overhead.	Generally higher overhead.
Isolation boundary	Operating-system level; depends on configuration and the shared kernel.	Hardware-virtualization level; often a stronger boundary.
Typical use	Microservices, web apps, testing and CI/CD.	Different operating systems and stronger workload separation.

Because containers do not need a full operating system for each application, they generally start faster and use fewer resources. Both technologies are useful; they simply solve isolation and deployment problems in different ways.

7. Applications and Conclusion

7.1 Common applications

Use case	How Docker helps
Web application deployment	Frontend and backend services can be packaged and deployed in a consistent environment.
Microservices	Each component can be built, deployed and scaled independently.
Software testing	Temporary environments can be created for tests and removed afterwards.
Continuous integration	Pipelines can build, test and publish images automatically.
Cloud computing	The same image runs across laptops, test servers and cloud platforms, and extra instances can be started when demand rises.

7.2 Conclusion

Docker solves the problem of differences between development and deployment environments. Docker images package an application and its dependencies, containers run those images, and container isolation — built on Linux namespaces and control groups — helps separate workloads that share the same host. Compared with virtual machines, containers have lower overhead because they share the host kernel, although this also means their security boundary differs and must be configured with care.

Used together, GitHub and Docker form a reliable workflow for developing, testing and delivering software. Docker also helps developers maintain consistent environments, simplify deployment and scale applications by running multiple instances when required.

> **REMEMBER**
>
> **GitHub shares the code, Docker packages the environment, and containers run the application in an isolated environment.**

References

[1] Docker, "What is Docker?" Docker Documentation. https://docs.docker.com/get-started/docker-overview/

[2] Docker, "What is a container?" Docker Documentation. https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

[3] Docker, "Docker Engine security." Docker Documentation. https://docs.docker.com/engine/security/

[4] Docker, "docker container run." Docker CLI Reference. https://docs.docker.com/reference/cli/docker/container/run/

[5] Kubernetes, "Containers." Kubernetes Documentation. https://kubernetes.io/docs/concepts/containers/

[6] Kubernetes, "Images." Kubernetes Documentation. https://kubernetes.io/docs/concepts/containers/images/